

Remote Procedure Call

Desarrollo de Software en Sistemas Distribuidos

Prácticas

- ▶ Grupos de 4 personas
- ▶ Organización de tareas con Trello o similar
- ▶ Proyecto integral en 3 etapas:
 - ▶ 1. gRPC
 - ▶ 2. Colas de mensajes: Kafka o RabbitMQ
 - ▶ 3. Web services: REST, GraphQL, SOAP(cliente)
- ▶ *Lo pendiente que quede en una entrega, se completa en la próxima.*

RPC: Características

- ▶ Es un mecanismo de comunicación en sistemas distribuidos.
- ▶ Los programas realizan la llamada de igual forma que a un programa local.
- ▶ Paradigma cliente/servidor.
- ▶ No existe memoria compartida entre el cliente y el servidor
 - ▶ ~~Pasaje de parámetros por referencia~~
 - ▶ Todo se pasa por valor

RPC: Tipos

► Texto

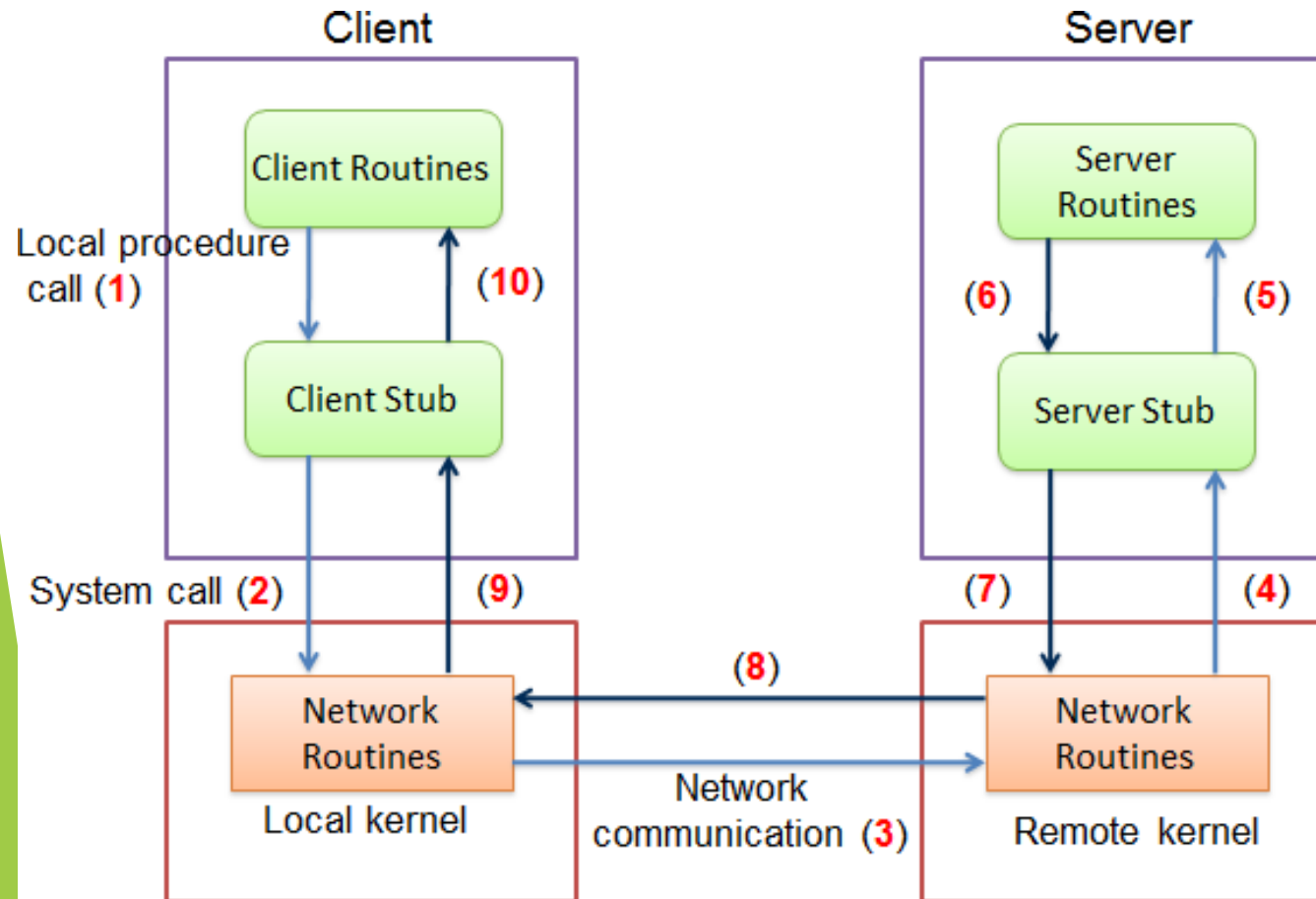
- Heterogeneidad.
- Garantiza la independencia entre los elementos de comunicación.
- No requiere de un protocolo especial.

► Binarios:

- Homogeneidad.
- Cliente y servidor deben “hablar el mismo idioma”.
- No requiere traducción.
- Transporte más veloz.

RPC: Esquema

- El esquema clásico del modelo RPC es el siguiente:



1. El cliente hace un llamado a un procedimiento local.
2. El stub empaqueta los datos necesarios para ser enviados por la red.
3. Se establece la comunicación con el servidor y se transporta el paquete.
4. El stub del servidor (skeleton) recibe los datos.
5. Se ejecuta el procedimiento remoto.
6. El skeleton recibe el resultado de la ejecución.
7. Empaqueta los datos para ser enviados por la red.
8. Los datos son transportados hacia el cliente a través de la red.
9. El stub del cliente recibe los datos.
10. El procedimiento local obtiene el resultado desempaquetado.

RPC: Stub cliente

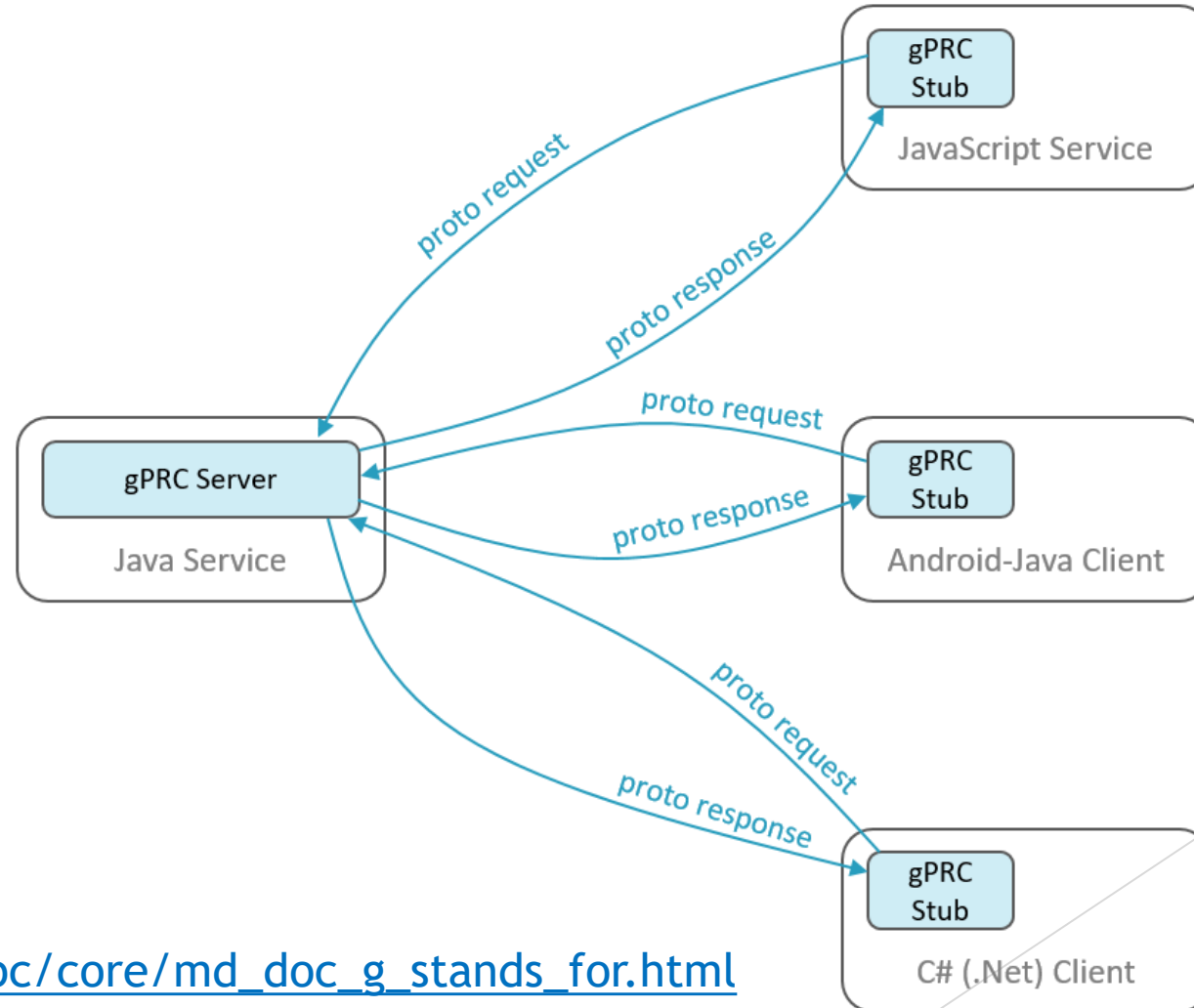
- ▶ La base del mecanismo RPC consiste en la introducción de “representantes” en el cliente y el servidor. En el lado cliente, el representante del servidor se denomina Stub:
 - ▶ Proporciona transparencia en la invocación del cliente.
 - ▶ Debe poseer llamadas con la misma declaración (forma) que el servidor.
 - ▶ El cliente invoca las llamadas del stub “como si fuese el servidor”.
 - ▶ Cada procedimiento que el cliente quiera invocar a través de RPCs necesita su propio stub.

RPC: Stub servidor

- ▶ En el lado servidor, el representante del cliente es un stub que también suele llamarse Skeleton:
 - ▶ Proporciona transparencia en el lado del servidor
 - ▶ Resuelve las llamadas invocadas desde el cliente
 - ▶ Devuelve al cliente el resultado del procedimiento.
 - ▶ Cada procedimiento que el servidor exporte a través de RPCs requiere su propio stub.

gRPC: g(versión) Remote Procedure Calls

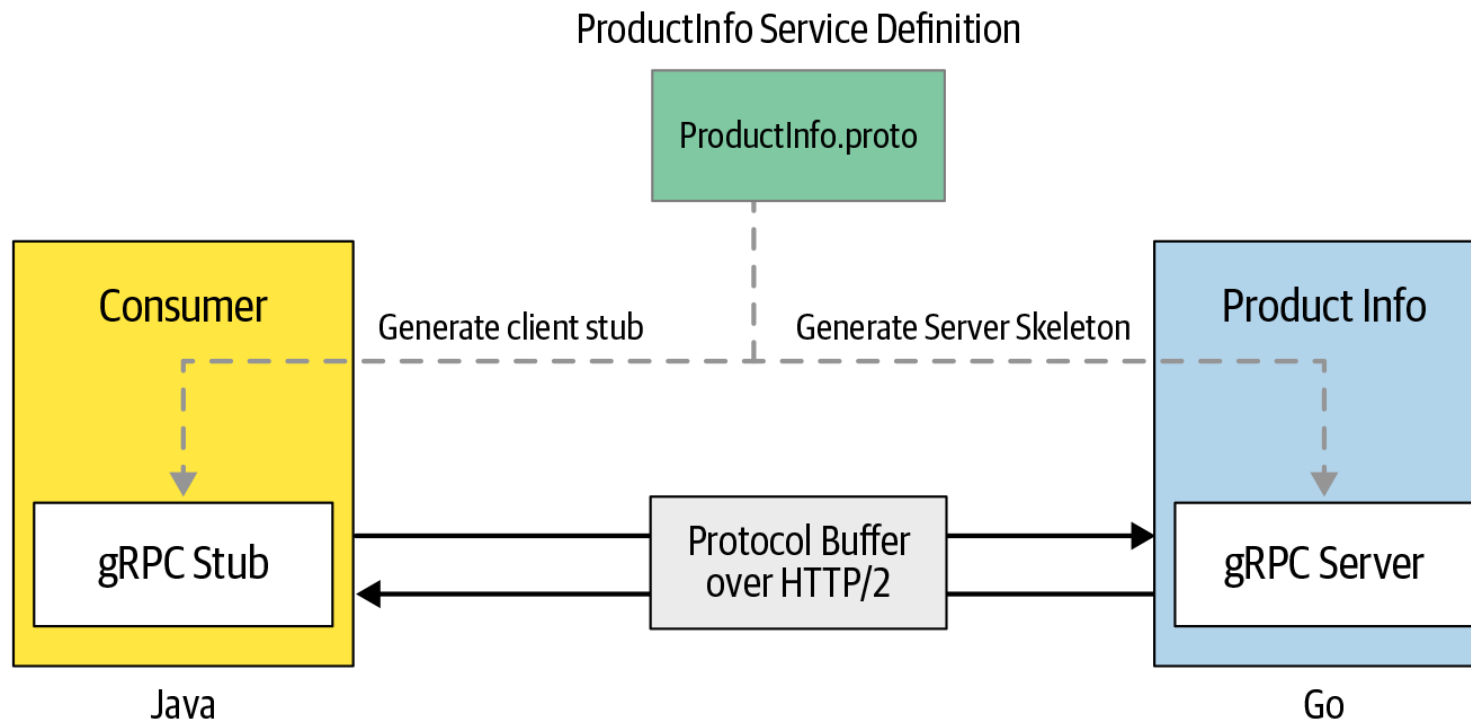
- C#
- C++
- Dart
- Go
- Java
- Kotlin
- Node
- Objective-C
- PHP
- Python
- Ruby



https://grpc.github.io/grpc/core/md_doc_g_stands_for.html

gRPC: Protobuf

- ▶ Abreviación de Protocol buffers
- ▶ Es un formato de serialización de datos en forma binaria.
- ▶ Al ser binario, se reduce el tamaño con respecto a otros formatos (JSON, XML).
- ▶ Los servicios son definidos en un archivo .proto mediante un lenguaje de descripción de interfaz (IDL)



gRPC: Ventajas y desventajas

Ventajas	Desventajas
Rendimiento y eficiencia (HTTP/2)	Compatibilidad con navegadores
Es bidireccional (streams)	Curva de aprendizaje
Seguridad integrada	Configuración inicial compleja
Soporte nativo de balanceo de carga	Mayor consumo de recursos para operaciones simples

gRPC

Documentación proto3: <https://developers.google.com/protocol-buffers/docs/proto3>

Lenguajes soportados y documentación de uso: <https://grpc.io/docs/languages/>

Ejemplo completo: <https://github.com/MarcosAmaro/hola-mundo-grpc>

Empresas que usan gRPC:

Netflix: <https://www.youtube.com/watch?v=r3m3kE8XEM4>

Uber: <https://www.uber.com/blog/ubers-next-gen-push-platform-on-grpc>

Dropbox: <https://dropbox.tech/infrastructure/courier-dropbox-migration-to-grpc>

Redhat: <https://www.redhat.com/en/blog/grpc-use-cases>

Linkedin: <https://www.linkedin.com/blog/engineering/infrastructure/linkedin-integrates-protocol-buffers-with-rest-li-for-improved-m>

Otras: <https://stackshare.io/grpc#stacks>